

3.3 深度神经网络

如何把网络“训深”：残差连接、归一化与权重初始化

Lecture Notes · 依据康奇宇老师课件整理

✓ 一页读懂本讲

本讲只回答一个问题：**网络加深以后为什么难训练，以及怎样才能训得动。**

- **问题**：深层网络的训练误差反而更高——这不是过拟合，而是**优化困难**；数学上的根源之一是反向传播中逐层连乘造成的**梯度消失**。
- **三条对策**，分别从“结构、中间表示、起点”三个角度下手：
 1. **残差连接**（改结构）：给梯度修一条不经过连乘的“高速公路”；
 2. **归一化**（管住中间表示）：让每层输入的尺度稳定，优化地形更平滑；
 3. **Xavier / He 初始化**（选好起点）：让信号方差在前向、反向传播中都不放大也不衰减。
- **历史**：AlexNet（8层）→ VGG（19层）→ GoogLeNet（22层）→ ResNet（152层），正是这些技巧把“深度”一步步变成了现实。

目录

1 训练困境：网络退化与梯度消失	2
1.1 现象：更深的网络，训练误差反而更高	2
1.2 回顾反向传播：梯度是一串连乘	3
1.3 缓解梯度消失的四条思路	3
2 训练策略一：残差连接	4
2.1 动机：深层模型“理应”不比浅层差	4
2.2 残差块	4
2.3 为什么有效：从梯度的角度看（补充）	5
2.4 ResNet 的整体结构	5
2.5 Bottleneck：用 1×1 卷积提高效率	5
3 训练策略二：归一化	6
3.1 出发点：优化地形	6
3.2 数据归一化：从输入层说起	7
3.3 批归一化 Batch Normalization (BN)	8
3.4 层归一化 Layer Normalization (LN)	10
3.5 RMS Normalization	10
3.6 Dynamic Tanh (DyT)：不要归一化层行不行？	10

3.7 实例归一化 Instance Normalization (IN)	11
3.8 组归一化 Group Normalization (GN)	12
3.9 小结：一个统一的视角	12
4 训练策略三：权重初始化	13
4.1 朴素的随机初始化	13
4.2 一个小实验：逐层观察激活值的统计量	13
4.3 Xavier 初始化 (Glorot & Bengio, 2010)	14
4.4 He 初始化及其数学推导 (He et al., 2015)	15
4.5 推广到 PReLU	16
4.6 小结	16
5 深度神经网络的发展历程	16
5.1 卷积网络的典型结构	16
5.2 ILSVRC：一条“越来越深”的主线	17
5.3 AlexNet (2012)	17
5.4 VGG (2014)	18
5.5 GoogLeNet (2014)	19
5.6 ResNet (2015) 与小结	20
6 课后习题与参考解答	20
7 课件勘误与补充说明	22
8 自测问题	23

1 训练困境：网络退化与梯度消失

1.1 现象：更深的网络，训练误差反而更高

直觉上，层数越多、参数越多，模型的表达能力越强，至少在**训练集**上应该拟合得更好。但 He 等人在 CIFAR-10 上的实验给出了相反的结果：56 层的普通 (plain) 卷积网络，训练误差和测试误差都高于 20 层的网络。〔PPT 第 4 页〕



图 1 56 层网络的训练误差（左）与测试误差（右）都高于 20 层网络。

⚠ 这不是过拟合

过拟合的特征是“训练误差低、测试误差高”。而这里**训练误差本身就更高**，说明深层模型连训练集都没学好，属于**欠拟合**。模型容量是足够的，问题出在**优化**上：越深的模型越难训练。这一现象也被称为网络的**退化**（degradation）。

1.2 回顾反向传播：梯度是一串连乘

考虑 L 层前馈网络 [PPT 第 5 页]：

$$\mathbf{x} = \mathbf{a}^{(0)} \rightarrow \mathbf{z}^{(1)} \rightarrow \mathbf{a}^{(1)} \rightarrow \dots \rightarrow \mathbf{z}^{(L)} \rightarrow \mathbf{a}^{(L)}, \quad \mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = f(\mathbf{z}^{(l)}).$$

记第 l 层的误差项为 $\mathbf{e}^{(l)} = \partial \mathcal{L} / \partial \mathbf{z}^{(l)}$ ，则

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{W}^{(l)}} = \mathbf{e}^{(l)} (\mathbf{a}^{(l-1)})^\top, \quad (1.1)$$

$$\mathbf{e}^{(l)} = \nabla f(\mathbf{z}^{(l)}) \odot \left((\mathbf{W}^{(l+1)})^\top \mathbf{e}^{(l+1)} \right). \quad (1.2)$$

式 (1.2) 说明：第 l 层的误差 = 后一层误差先经权重矩阵“传回来”，再逐元素乘以本层激活函数的导数。（课件把误差写成行向量，故记作 $\mathbf{e}^{(l+1)} \mathbf{W}^{(l+1)}$ ，含义相同。）

对 (1.2) 取范数并逐层递推：

$$\begin{aligned} \|\mathbf{e}^{(l)}\|_2 &\leq \|\nabla f(\mathbf{z}^{(l)})\|_2 \cdot \|\mathbf{W}^{(l+1)}\|_2 \cdot \|\mathbf{e}^{(l+1)}\|_2 \\ &\leq \underbrace{\prod_{i=l}^{L-1} \left(\|\nabla f(\mathbf{z}^{(i)})\|_2 \cdot \|\mathbf{W}^{(i+1)}\|_2 \right)}_{\text{每个因子} < 1 \text{ 时, 连乘趋于 } 0} \cdot \left\| \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(L)}} \right\|_2. \end{aligned} \quad (1.3)$$

梯度消失 (Gradient Vanishing)

神经网络层数过多时，由于链式法则，反向传播过程中**浅层（靠近输入的层）**参数的导数趋近于 0，无法得到有效的更新和学习。

✎ 数值感受

假设每一层“激活函数导数 × 权重”的绝对值都略小于 1，取 0.9。经过 30 层传播后 $0.9^{30} \approx 0.0424$ ，浅层收到的梯度只剩约 4%；若取 0.5，30 层后约为 10^{-9} ，基本等于没有梯度。反过来，若每个因子都大于 1，连乘会指数增长，对应**梯度爆炸**。两者是同一个问题的两面。

1.3 缓解梯度消失的四条思路

观察上界 (1.3)，想让连乘不塌缩，可以从不同的“因子”下手 [PPT 第 6 页]：

1. **更换激活函数** (管 ∇f)。Sigmoid 的导数 $\sigma'(x) = \sigma(x)(1 - \sigma(x)) \in (0, \frac{1}{4}]$, $\tanh$ 的导数 $1 - \tanh^2(x) \in (0, 1]$, 都不超过 1, 且在饱和区接近 0; ReLU 在正半轴导数恒为 1, Leaky ReLU 在负半轴也保留一个小斜率, 可有效缓解梯度消失。
2. **残差连接** (改连乘结构): ResNet (He et al., 2016), 见第 2 节。
3. **归一化** (管每层输入的尺度, 间接管 ∇f 与 W 的有效大小), 见第 3 节。
4. **合适的初始化** (管 W 的初始尺度), 见第 4 节。

2 训练策略一：残差连接

2.1 动机：深层模型“理应”不比浅层差

一个构造性的论证 [PPT 第 8 页]：给定一个训练好的浅层模型，构造深层模型时——

- 把浅层模型的所有层**拷贝**到深层模型的低层；
- 其余新增的高层全部设为**恒等映射** (identity mapping)。

这样得到的深层模型与浅层模型输出完全相同，性能相当。因此深层模型的**最优解**不会比浅层差。

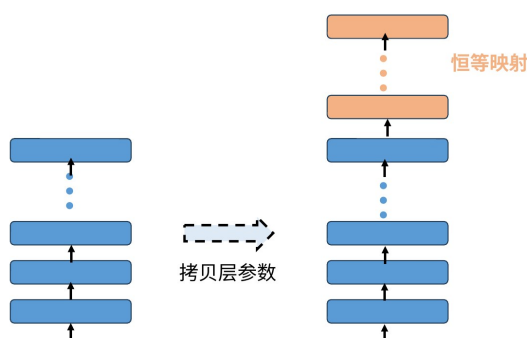


图 2 构造性论证：拷贝浅层参数，新增层取恒等映射。

💡 从“解存在”到“解好找”

实验中深层网络更差，说明优化器**找不到**这个解——让一叠非线性层去逼近恒等映射，其实并不容易。于是思路变成：**直接在结构里加一条“直连边”，让恒等映射成为默认选项**，网络只需学习“在恒等映射之上再修正多少”。

2.2 残差块

假设我们希望用一个非线性单元 $F(x, \theta)$ (一层或多层卷积) 去逼近目标函数 $h(x)$ 。把目标函数拆成两部分 [PPT 第 9 页]：

$$h(x) = \underbrace{x}_{\text{恒等函数}} + \underbrace{(h(x) - x)}_{\text{残差函数} \approx F(x, \theta)} . \quad (2.1)$$

网络不再直接拟合 $h(x)$ ，而是让卷积层 $F(x, \theta)$ 去拟合**残差** $h(x) - x$ ，输出为 $F(x, \theta) + x$ 。如果最优映射接近恒等，只需把 F 的权重推向 0 即可，这比让多层非线性堆叠去拟合恒等映射容易得多。

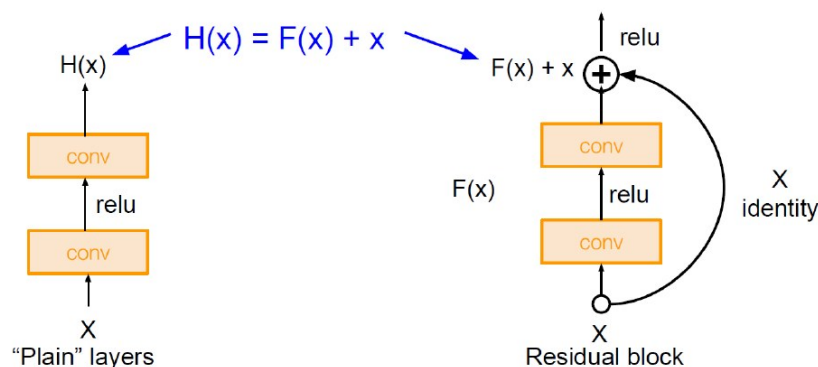


图3 普通堆叠层（左）与残差块（右）。残差块输出 $H(x) = F(x) + x$ 。

2.3 为什么有效：从梯度的角度看（补充）

忽略相加之后的 ReLU，第 l 个残差块写作 $x_{l+1} = x_l + F(x_l, \theta_l)$ 。逐块展开得到

$$x_L = x_l + \sum_{i=l}^{L-1} F(x_i, \theta_i), \quad \frac{\partial \mathcal{L}}{\partial x_l} = \frac{\partial \mathcal{L}}{\partial x_L} \left(I + \frac{\partial}{\partial x_l} \sum_{i=l}^{L-1} F(x_i, \theta_i) \right).$$

与式 (1.3) 的连乘对比，这里多了一个单位阵 I ：深层的梯度 $\partial \mathcal{L} / \partial x_L$ 可以原封不动地传到任意浅层，其余部分是“加”上去的修正项。只要修正项不恰好等于 $-I$ ，梯度就不会消失。

✓ 核心结论

残差连接把梯度传播从“连乘”改成了“恒等项 + 修正项”，这就是它能训练上百层网络的关键。

2.4 ResNet 的整体结构

- ILSVRC 2015 冠军：152 层，top-5 错误率 3.57%；在 ILSVRC'15 与 COCO'15 的分类、检测等全部赛道夺冠。[PPT 第 10 页]
- 主体是残差块的堆叠，每个基本残差块含两个 3×3 卷积层。
- 开头一个卷积层 ($7 \times 7, 64$, 步幅 2)；每进入新阶段，通道数翻倍、同时用步幅 2 把空间尺寸减半（如 “ 3×3 conv, 128, /2”）。
- 末尾用全局平均汇聚，除输出 1000 类的 FC 层外没有其他全连接层。

2.5 Bottleneck：用 1×1 卷积提高效率

1×1 卷积的含义 [PPT 第 11 页]：对 $56 \times 56 \times 64$ 的特征图使用 32 个 1×1 卷积，每个卷积核大小为 $1 \times 1 \times 64$ ，在每个空间位置上对 64 维通道向量做一次内积，输出 $56 \times 56 \times 32$ 。空间尺寸不变，只改变通道数（深度），相当于逐位置的全连接层，可用来降维或升维。

Bottleneck 残差块 [PPT 第 12 页] (用于 ResNet-50/101/152), 输入 $28 \times 28 \times 256$:

$$\underbrace{1 \times 1, 64}_{\text{降维到 64 通道}} \rightarrow \underbrace{3 \times 3, 64}_{\text{只在 64 通道上做空间卷积}} \rightarrow \underbrace{1 \times 1, 256}_{\text{升维回 256 通道}} \oplus x.$$

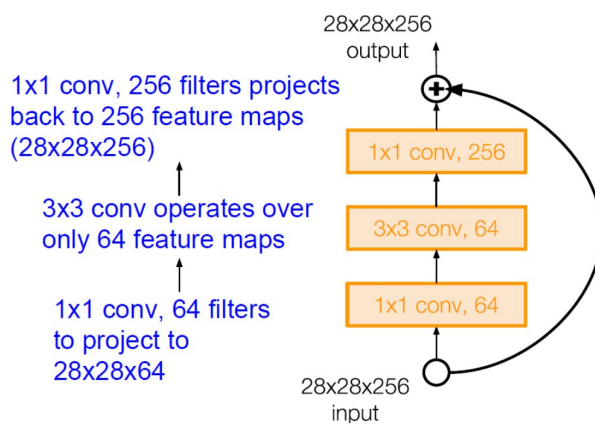


图 4 Bottleneck 残差块：“先压缩、再卷积、后还原”。

✎ 参数量对比 (忽略偏置)

- Bottleneck: $1 \cdot 1 \cdot 256 \cdot 64 + 3 \cdot 3 \cdot 64 \cdot 64 + 1 \cdot 1 \cdot 64 \cdot 256 = 16,384 + 36,864 + 16,384 = \mathbf{69,632}$;
- 直接在 256 通道上堆两个 3×3 卷积: $2 \times (3 \cdot 3 \cdot 256 \cdot 256) = \mathbf{1,179,648}$ 。

参数量与计算量都降到约 1/17, 省下的预算可以用来把网络做得更深。

3 训练策略二：归一化

3.1 出发点：优化地形

优化地形 (Optimization Landscape)

在高维参数空间中, 损失函数的曲面形状。

Li 等人 (NeurIPS 2018) 把损失曲面可视化后发现 [PPT 第 14–15 页]: 不带跳连的深层网络, 地形崎岖、混乱, 充满尖锐的沟壑, 且越深越严重; 而 ResNet-56、DenseNet-121 这类带跳连的网络, 地形平滑得多, 近似一个“碗”。

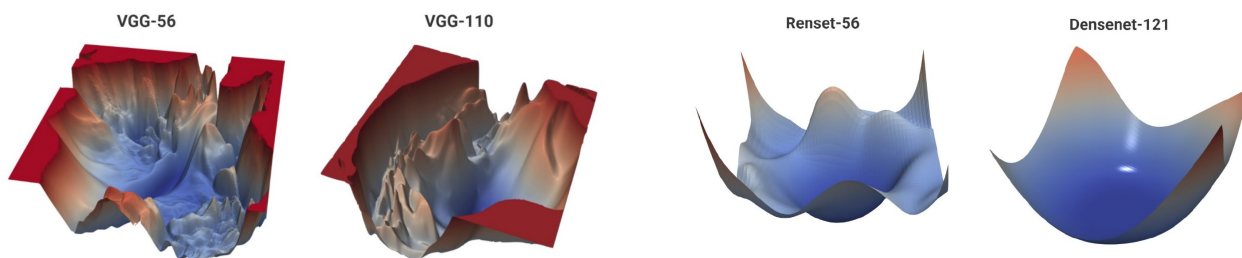


图5 左：无跳连的深层网络，地形崎岖；右：ResNet-56 与 DenseNet-121，地形平滑。

✓ 核心结论

好的优化地形通常比较**平滑**，更容易优化。ReLU 激活函数、残差连接、逐层归一化，都可以理解为“**通过修改网络结构来获得更好的优化地形**”。

3.2 数据归一化：从输入层说起

为什么特征的尺度会影响优化？[PPT 第 16-17 页] 考虑 $\hat{y} = b + w_1x_1 + w_2x_2$, $\mathcal{L} = (y - \hat{y})^2$, 则

$$\frac{\partial \mathcal{L}}{\partial w_j} = -2(y - \hat{y})x_j.$$

若 $x_1 \in \{1, 2, \dots\}$ 而 $x_2 \in \{100, 200, \dots\}$, 那么 w_2 方向的梯度比 w_1 方向大约两个数量级：损失等高线是**狭长的椭圆**，梯度方向不指向最小值，梯度下降来回震荡，学习率只能迁就最陡的方向。把两个特征缩放到相同尺度后，等高线接近**正圆**，负梯度直指圆心，收敛快得多。

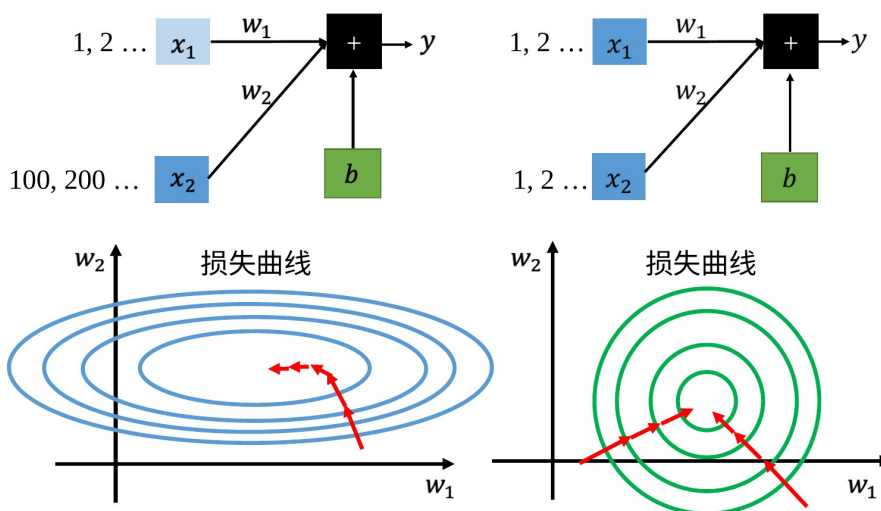


图6 特征尺度不同（左）与相同（右）时的损失等高线和梯度下降轨迹。

设有 n 个样本 $x^1, \dots, x^n$, 对**每个维度** d 独立处理，常用两种方式：

	标准化 (standardization) [PPT 第 18 页]	最大最小值归一化 [PPT 第 19 页]
统计量	$m_d = \frac{1}{n} \sum_{i=1}^n x_d^i, \quad \sigma_d^2 = \frac{1}{n} \sum_{i=1}^n (x_d^i - m_d)^2$	$\max_d, \min_d$
变换	$x_d^i \leftarrow \frac{x_d^i - m_d}{\sigma_d}$	$x_d^i \leftarrow \frac{x_d^i - \min_d}{\max_d - \min_d}$
结果	每个维度均值为 0、方差为 1	每个维度落在 $[0, 1]$

⚠ 注意

统计量只能在**训练集**上计算，测试时沿用同一组统计量。最大最小值归一化对离群点敏感。

3.3 批归一化 Batch Normalization (BN)

3.3.1 动机：内部协变量偏移

对第 l 层，净输入为 $\mathbf{z}^{(l)}$ ，输出为 $\mathbf{a}^{(l)} = f_l(\mathbf{z}^{(l)}) = f_l(\mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$ 。 [PPT 第 20 页]

内部协变量偏移 (Internal Covariate Shift)

每一层的参数在更新过程中，会改变下一层输入的分布；神经网络层数越多，这种偏移表现得越明显。

输入层的数据分布是固定的，做一次数据归一化就够了；但隐藏层的“输入”会随着前面各层参数的更新不断漂移。**逐层归一化**的思想就是把数据归一化搬到网络内部：使每一层神经元的分布在训练过程中保持一致。

3.3.2 算法

设 mini-batch 大小为 K ，该层净输入为 $\mathbf{z}_1, \dots, \mathbf{z}_K$ 。

第 1 步：独立计算该层**每个维度**在 batch 上的均值和方差：

$$\mu_B = \frac{1}{K} \sum_{k=1}^K z_k, \quad \sigma_B^2 = \frac{1}{K} \sum_{k=1}^K (z_k - \mu_B) \odot (z_k - \mu_B). \quad (3.1)$$

第 2 步：归一化（实现时分母加小常数 ϵ 防止除零）：

$$\hat{z}_k = \frac{z_k - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}. \quad (3.2)$$

第 3 步：缩放与平移 [PPT 第 22 页]：

$$\tilde{z}_k = \hat{z}_k \odot \gamma + \beta, \quad \text{此时 } \mathbb{E}[\tilde{z}_k] = \beta, \quad \text{Var}[\tilde{z}_k] = \gamma^2. \quad (3.3)$$

💡 既然又加回均值和方差，归一化岂不是白做了？

把每个单元强行标准化为“0 均值、1 方差”会降低该单元的表达能Ⓕ力（例如 Sigmoid 的输入被限制在近似线性的区域）。引入可学习的 γ, β 后，单元的净输入又可以有任意均值和方差，表达能力得以恢复。但新旧参数化的学习动态完全不同：

- **旧参数化**：均值和方差取决于低层神经网络参数之间的复杂关联，牵一发而动全身；
- **新参数化**：均值和方差只由 β, γ 决定，解除了与下层计算的密切耦合，可以直接通过梯度下降来学习。

3.3.3 BN 放在哪里

典型用法是 FC（或 Conv） $\rightarrow$ BN $\rightarrow$ 非线性激活，即 BN 作用在仿射变换之后、非线性激活之前的净输入 z 上 [PPT 第 21 页]。理由：每层的输入 $a^{(l-1)}$ 通常是前一层非线性激活的输出，分布往往是非高斯的（如 ReLU 输出非负且偏斜），不适合做标准化；而仿射变换的输出 z 是大量项之和，更接近高斯分布。

3.3.4 训练与测试的区别

测试时往往是单个样本输入，没有 batch 可言，也不希望一个样本的预测受同批其他样本影响 [PPT 第 24 页]。因此：

- 均值和方差不在测试 batch 上计算，而是使用训练时追踪到的、整个训练集上的统计量；
- 实践中用移动平均在线估计： $\mu_{\text{run}} \leftarrow (1 - \rho) \mu_{\text{run}} + \rho \mu_B$ （方差同理）；
- 测试时 BN 退化为固定的逐维仿射变换，可以对单一样本评估，也可以合并进前面的线性层。

⚠ 常见 bug

推理时忘记切换到评估模式（如 PyTorch 的 `model.eval()`），BN 仍使用当前 batch 的统计量，结果会异常。

3.3.5 作用与局限

作用 [PPT 第 23 页]：改善梯度的传播；允许使用比较大的学习率；减小对初始化的依赖。

局限：batch size 不能太小，否则 batch 统计量噪声过大，性能明显下降；训练与测试行为不一致；不适合变长的序列数据（见 3.4 小节）。

3.3.6 全连接网络与卷积网络中的 BN

	全连接网络	卷积网络
输入 x	$N \times D$	$N \times C \times H \times W$
μ, σ	$1 \times D$ （对 N 求平均）	$1 \times C \times 1 \times 1$ （对 N, H, W 求平均）
γ, β	$1 \times D$	$1 \times C \times 1 \times 1$

卷积网络中，同一通道的所有空间位置共享同一个卷积核，被视为“同一个特征”的多次观测，因此每个通道共用一组统计量和一组 γ, β 。[PPT 第 25 页]

3.4 层归一化 Layer Normalization (LN)

对 $N \times D$ 的输入，BN 沿 **batch 维**（对每个特征、跨样本）求统计量；LN 沿**特征维**（对每个样本、跨该层所有神经元）求统计量， μ, σ 形状为 $N \times 1$ ，而 γ, β 仍为 $1 \times D$ 。[PPT 第 26 页]

特点：每个样本只用自己的统计量，与 batch 无关，因此**训练和测试时操作一致**，不需要移动平均；常用于循环神经网络，也是 Transformer 的标准配置。

为什么 BN 不适合序列数据

一个 batch 中的三句话 [PPT 第 27 页]：

- Hello , How do you do ? (7 个 token)
- Nice to meet you . <pad> <pad> (5 个 token + 2 个 pad)
- I love baseball . <pad> <pad> <pad> (4 个 token + 3 个 pad)

BN 要在“同一位置、跨样本”上估计统计量。在句末的位置上，只有第一句是真实 token，其余都是填充，估计值很不合理；再加上 batch size 较小时统计量本身就不准。LN 只在同一层的不同神经元之间做归一化，没有上述问题。

3.5 RMS Normalization

Root Mean Square Layer Normalization (Zhang & Sennrich, 2019)。[PPT 第 28 页] 记某层的 n 个神经元为 $a_1, \dots, a_n$ ， g_i, b_i 为可学习参数：

$$\text{Layer Norm: } \bar{a}_i = \frac{a_i - \mu}{\sigma} g_i + b_i, \quad \mu = \frac{1}{n} \sum_{i=1}^n a_i, \quad \sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (a_i - \mu)^2}; \quad (3.4)$$

$$\text{RMS Norm: } \bar{a}_i = \frac{a_i}{\sqrt{\frac{1}{n} \sum_{i=1}^n a_i^2}} g_i + b_i. \quad (3.5)$$

RMS Norm 是 LN 的简化版本：**去掉“减均值” (re-centering)**，只保留**“除以尺度” (re-scaling)**。原论文认为 LN 的收益主要来自缩放不变性；省去均值计算后计算量更小（论文报告不同模型上运行时间减少约 7%–64%），效果基本持平。当前主流大语言模型（如 LLaMA 2、Qwen 2 等）都采用 RMS Norm，且实现中通常连偏置 b_i 也省去。

3.6 Dynamic Tanh (DyT)：不要归一化层行不行？

“Transformers without Normalization” (Zhu, Chen, He, LeCun, Liu, CVPR 2025)。[PPT 第 29–30 页]

观察：把训练好的 ViT、wav2vec 2.0、DiT 中各个 LN 层的“输入—输出”画成散点图，浅层近似一条直线，而**深层呈现 S 形曲线，很像 tanh**。原因在于：LN 对每个 token 而言是线性的，但不同 token 的

标准差不同；少数幅值极大的输入会被除以一个很大的标准差，于是从整体看，极端值被“压扁”了。

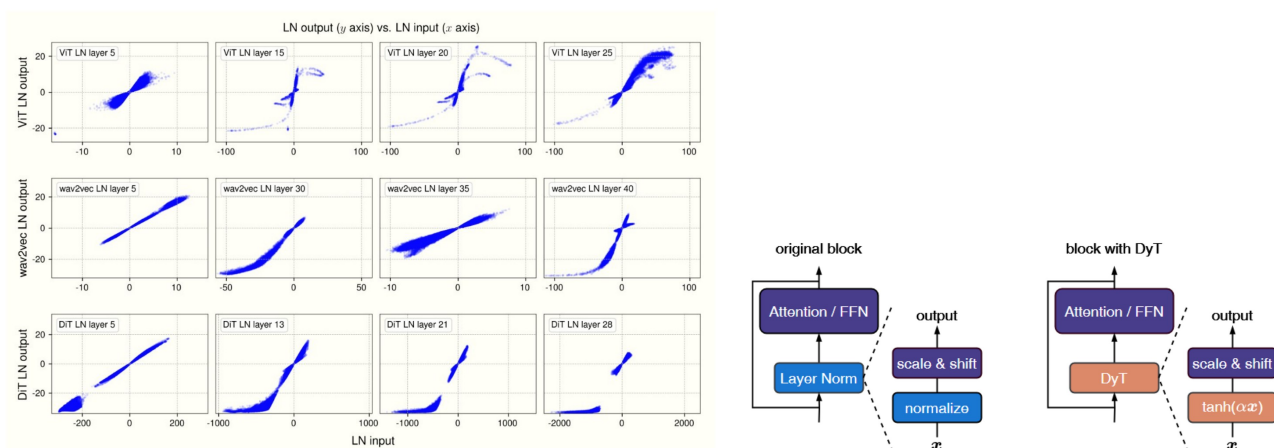


图7 左：三种 Transformer 中 LN 层的输入—输出关系；右：用 DyT 直接替换 LN。

做法：既然 LN 的整体效果像一个 S 形挤压函数，就直接用一个逐元素函数替换它：

$$\text{DyT}(x) = \gamma \odot \tanh(\alpha x) + \beta, \quad (3.6)$$

其中 α 是可学习的标量， γ, β 与 LN 中的缩放、平移参数相同。DyT 不需要计算任何统计量，却能在多种 Transformer 任务上达到与 LN 相当的效果。这提示我们：归一化层的核心作用之一，也许就是对极端激活值做非线性压缩。

3.7 实例归一化 Instance Normalization (IN)

定义 [PPT 第 31, 35 页]：对 $N \times C \times H \times W$ 的输入， μ, σ 形状为 $N \times C \times 1 \times 1$ ，即每个样本的每个通道单独在空间维 (H, W) 上做归一化； γ, β 仍为 $1 \times C \times 1 \times 1$ 。

来源：IN (Ulyanov et al., 2016) 最初用于图像风格迁移。[PPT 第 33–34 页]

- “内容”：图像中的轮廓、结构，如房子、山、水、树的位置；“风格”：颜色、纹理、笔触、明暗对比等“视觉气质”。
- CNN 中间层 feature map 的每个 channel 可看作对某类纹理/图案的响应。**均值**表示该纹理在这张图中出现的“强度”，**方差**表示该纹理在图中变化的“程度”。
- 因此，**各通道的均值和方差携带的正是风格信息**。IN 逐样本、逐通道地抹去它们，就去掉了原图的风格；再赋予新的均值和方差，就能生成不同风格的图像（后续工作 AdaIN 正是直接用风格图的统计量替换内容图的统计量）。

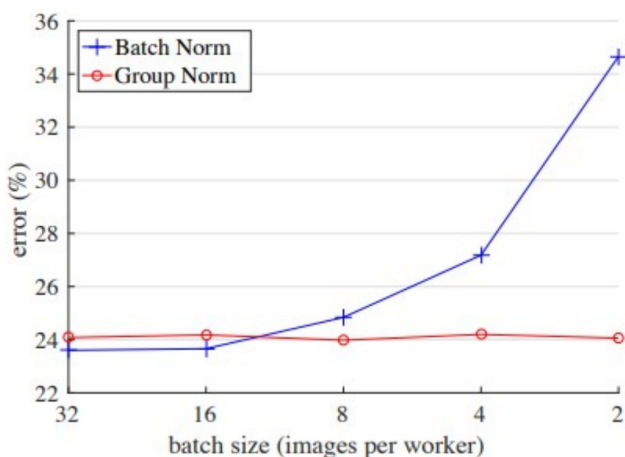


图8 风格迁移：内容图 + 风格图 = 保留内容、更换风格。

3.8 组归一化 Group Normalization (GN)

动机 [PPT 第 36–37 页]：图像分割、检测、视频等任务占用显存很大，batch size 往往只能是个位数，此时 BN 对均值和标准差的估计很不精确。

做法 (Wu & He, 2018)：将每个样本 feature map 的 C 个通道分成 G 组，每组 C/G 个通道，在每组内对所有元素 ($C/G \times H \times W$ 个) 求均值和标准差来归一化。GN 同样**独立于 batch**，是 LN 与 IN 的折中： $G = 1$ 时退化为 LN， $G = C$ 时退化为 IN。



观察 (ImageNet, ResNet-50)：

- batch size 较大 (32、16) 时，BN 略好于 GN；
- batch size 较小 (8、4、2) 时，GN 明显好于 BN：BN 的错误率从约 23.6% 升到约 34.7%，而 GN 稳定在 24% 左右。

图 9 BN 与 GN 的错误率随 batch size 的变化。

3.9 小结：一个统一的视角

所有归一化层都是同一个公式 $y = \gamma \frac{x - \mu}{\sigma} + \beta$ ，区别只在于 μ, σ 在张量的哪些维度上计算。[PPT 第 32, 37 页]

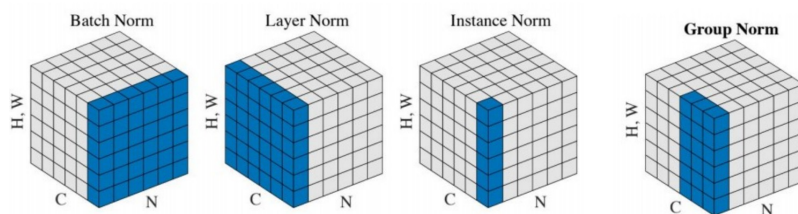


图 10 蓝色区域表示共用同一组 μ, σ 的元素 (N ：样本； C ：通道； H, W ：空间)。

方法	求统计量的维度	μ, σ 形状	依赖 batch	典型场景
BN	N, H, W	$1 \times C \times 1 \times 1$	是	CNN 图像分类，batch 较大
LN	C, H, W	$N \times 1 \times 1 \times 1$	否	RNN、Transformer
RMS Norm	同 LN (不减均值)	—	否	大语言模型
IN	H, W	$N \times C \times 1 \times 1$	否	风格迁移、图像生成
GN	组内 $C/G, H, W$	$N \times G \times 1 \times 1$	否	分割、检测等小 batch 任务
DyT	不计算统计量	—	否	替换 Transformer 中的 LN

4 训练策略三：权重初始化

4.1 朴素的随机初始化

两种常见做法 [PPT 第 40 页]：

- **高斯分布初始化**：参数从固定均值（如 0）、固定方差（如 0.01）的高斯分布中采样，
 $W = \text{sigma} * \text{np.random.randn}(\text{fan_in}, \text{fan_out});$
- **均匀分布初始化**：参数在区间 $[-r, r]$ 内均匀采样，
 $W = 2r * \text{np.random.rand}(\text{fan_in}, \text{fan_out}) - r。$

$[-r, r]$ 上均匀分布的方差为 $r^2/3$ ，所以若希望方差为 σ^2 ，应取 $r = \sqrt{3\sigma^2}$ 。

⚠ 注意

固定方差的随机初始化在浅层网络上效果不错，但在深层网络上有较大问题。此外，不能把权重全部初始化为 0（或相同的值）：同一层的神经元将收到完全相同的梯度，永远无法分化（对称性问题）。

4.2 一个小实验：逐层观察激活值的统计量

设置 [PPT 第 41–45 页]：10 层全连接网络，每层 500 个神经元，输入为标准高斯，逐层记录激活值的标准差。下面的代码可以直接复现课件中的现象。

```
1 import numpy as np
2 L, M, N = 10, 500, 1000                                # layers, width, samples
3 def run(act, scale):
4     a = np.random.randn(N, M); stds = []
5     for l in range(L):
6         W = np.random.randn(M, M) * scale(M, M)
7         z = a @ W
8         a = np.tanh(z) if act == "tanh" else np.maximum(z, 0)
9         stds.append(a.std())
10    return stds
11 run("tanh", lambda i, o: 0.01)                          # small random
12 run("tanh", lambda i, o: 1.0)                            # large random
13 run("tanh", lambda i, o: np.sqrt(2 / (i + o)))           # Xavier
14 run("relu", lambda i, o: np.sqrt(2 / (i + o)))           # Xavier + ReLU
15 run("relu", lambda i, o: np.sqrt(2 / i))                # He + ReLU
```

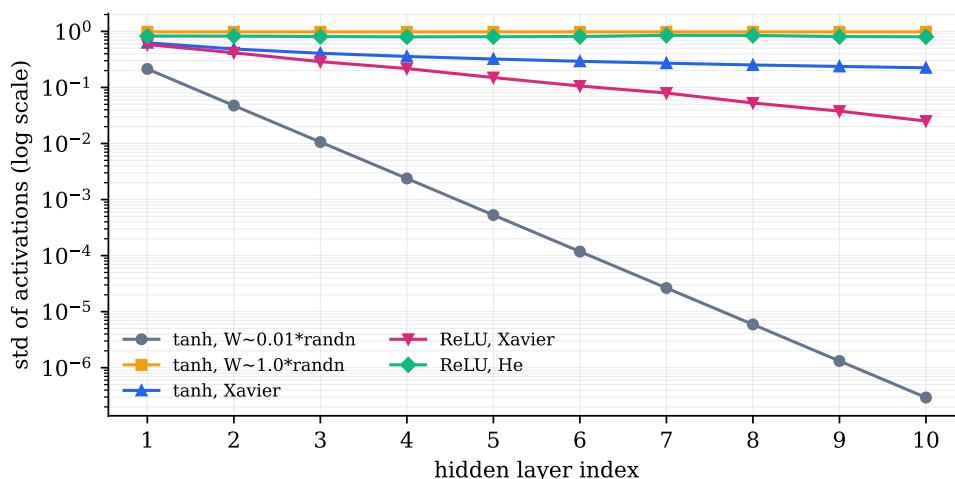


图 11 五种设置下各隐藏层激活值的标准差（纵轴为对数坐标；笔者按课件设置复现）。

设置	现象	后果
tanh, $\sigma = 0.01$	标准差每层缩小到约 0.22 倍，第 10 层 $\approx 10^{-7}$	所有激活输出都变成 0；由 (1.1), $\partial \mathcal{L} / \partial \mathbf{W}^{(l)} = \mathbf{e}^{(l)} (\mathbf{a}^{(l-1)})^\top \approx \mathbf{0}$
tanh, $\sigma = 1$	标准差 ≈ 0.98 ，输出集中在 ± 1	激活函数饱和， $\tanh'(z) = 1 - a^2 \approx 0$ ，梯度为 0
tanh, Xavier	标准差从 0.63 缓慢降到 0.22，分布形状良好	可以正常训练
ReLU, Xavier	标准差每层缩小到约 0.71 倍	当使用 ReLU 时，方差又逐渐接近 0
ReLU, He	标准差稳定在 0.8 左右	可以正常训练

第一行的 0.22 可以用第 4.4 小节的公式直接验证： $\sqrt{M \cdot \text{Var}[w]} = \sqrt{500 \times 10^{-4}} \approx 0.224$ ；第四行：Xavier 在 $M_{l-1} = M_l = 500$ 时 $\text{Var}[w] = 1/500$ ，ReLU 再砍掉一半，每层标准差乘以 $\sqrt{1/2} \approx 0.71$ 。

✓ 核心结论

初始化的核心不在于“随机”，而在于**尺度**：太小则信号逐层消失，太大则激活饱和。合适的尺度应当由**层的宽度**决定。

4.3 Xavier 初始化 (Glorot & Bengio, 2010)

$$\text{Var}[w^{(l)}] = \frac{2}{M_{l-1} + M_l} \iff w \sim \mathcal{N}\left(0, \frac{2}{M_{l-1} + M_l}\right) \text{ 或 } w \sim \mathcal{U}\left(-\sqrt{\frac{6}{M_{l-1} + M_l}}, \sqrt{\frac{6}{M_{l-1} + M_l}}\right), \quad (4.1)$$

其中 M_{l-1}, M_l 分别是该层的输入、输出维度 (fan-in, fan-out)。代码：`W = np.random.randn(M_prev, M_cur) * np.sqrt(2 / (M_prev + M_cur))`。 [PPT 第 43 页]

Xavier 初始化的推导假设激活函数在 0 附近近似线性（适用于 tanh），推导过程见第 6 节习题 1。对 ReLU，这个假设不成立：ReLU 把一半的输入置零，信号的二阶矩每层减半 [PPT 第 44 页]，这正是 He 初始化要修正的地方。

4.4 He 初始化及其数学推导 (He et al., 2015)

思路: 保持前向传播中不同层间**激活值**的方差相同, 保持反向传播中不同层间**梯度**的方差相同。[PPT 第 46 页]

设定与假设:

- 第 l 层: 仿射变换 $\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$, 非线性变换 $\mathbf{a}^{(l-1)} = \text{ReLU}(\mathbf{z}^{(l-1)})$;
- $\mathbf{W}^{(l)}$ 的每个元素独立同分布, $\mathbf{a}^{(l-1)}$ 的每个元素也独立同分布, 且 $\mathbf{W}^{(l)}$ 与 $\mathbf{a}^{(l-1)}$ 相互独立;
- $\mathbb{E}[w^{(l)}] = 0$, 且 w 的分布关于 0 对称; 偏置初始化为 $\mathbf{b}^{(l)} = \mathbf{0}$ 。

第 1 步: 把和的方差拆开。 第 i 个净输入为 $z_i^{(l)} = \sum_{d=1}^{M_{l-1}} W_{i,d}^{(l)} a_d^{(l-1)}$, 是 M_{l-1} 个独立同分布项之和。利用 $\text{Var}[x_1 + \cdots + x_n] = \text{Var}[x_1] + \cdots + \text{Var}[x_n]$,

$$\text{Var}[z^{(l)}] = M_{l-1} \text{Var}[w^{(l)} a^{(l-1)}]. \quad (4.2)$$

第 2 步: 乘积的方差。 由 $\mathbb{E}[w^{(l)}] = 0$ 及独立性, $\mathbb{E}[w^{(l)} a^{(l-1)}] = \mathbb{E}[w^{(l)}] \mathbb{E}[a^{(l-1)}] = 0$, 于是 [PPT 第 47 页]

$$\begin{aligned} \text{Var}[w^{(l)} a^{(l-1)}] &= \mathbb{E}[(w^{(l)} a^{(l-1)})^2] - (\mathbb{E}[w^{(l)} a^{(l-1)}])^2 = \mathbb{E}[(w^{(l)})^2] \mathbb{E}[(a^{(l-1)})^2] \\ &= (\mathbb{E}[(w^{(l)})^2] - \mathbb{E}^2[w^{(l)}]) \mathbb{E}[(a^{(l-1)})^2] = \text{Var}[w^{(l)}] \mathbb{E}[(a^{(l-1)})^2]. \end{aligned} \quad (4.3)$$

因此

$$\text{Var}[z^{(l)}] = M_{l-1} \text{Var}[w^{(l)}] \mathbb{E}[(a^{(l-1)})^2]. \quad (4.4)$$

⚠ 注意

这里出现的是二阶矩 $\mathbb{E}[(a^{(l-1)})^2]$, 而**不是**方差 $\text{Var}[a^{(l-1)}]$: ReLU 的输出非负, 均值不为 0, 两者并不相等。这正是 He 初始化与 Xavier 初始化的分歧所在。

第 3 步: 处理 ReLU。 由 $b^{(l-1)} = 0$ 与 $\mathbb{E}[w^{(l-1)}] = 0$, 有 $\mathbb{E}[z^{(l-1)}] = 0$; 又因 $w^{(l-1)}$ 的分布关于 0 对称, $z^{(l-1)}$ 的分布也关于 0 对称。于是 [PPT 第 48 页]

$$\begin{aligned} \mathbb{E}[(a^{(l-1)})^2] &= \int_{-\infty}^{+\infty} (\text{ReLU}(z))^2 p(z) dz = \int_0^{+\infty} z^2 p(z) dz \\ &\stackrel{\text{对称性}}{=} \frac{1}{2} \int_{-\infty}^{+\infty} z^2 p(z) dz = \frac{1}{2} \mathbb{E}[(z^{(l-1)})^2] = \frac{1}{2} \text{Var}[z^{(l-1)}]. \end{aligned} \quad (4.5)$$

第 4 步: 得到递推并令方差守恒。 把 (4.5) 代入 (4.4):

$$\text{Var}[z^{(l)}] = \frac{M_{l-1}}{2} \text{Var}[w^{(l)}] \text{Var}[z^{(l-1)}] \xrightarrow{\text{Var}[z^{(l)}] = \text{Var}[z^{(l-1)}]} \boxed{\frac{M_{l-1}}{2} \text{Var}[w^{(l)}] = 1} \quad (4.6)$$

即 $\text{Var}[w^{(l)}] = 2/M_{l-1}$, 代码: `W = np.random.randn(M_prev, M_cur) * np.sqrt(2 / M_prev)`。
[PPT 第 45 页]

反向传播：类似分析可得 $\frac{M_l}{2} \text{Var}[w^{(l)}] = 1$ 。He 等人指出，两个条件任选其一即可：若按前向条件初始化，反向信号的总缩放因子为 $\prod_l M_l/M_{l-1}$ ，相邻项相消后只与首末两层的宽度之比有关，不会随深度指数变化。

💡 与 Xavier 的关系

在 $M_{l-1} = M_l = M$ 时，Xavier 给出 $\text{Var}[w] = 1/M$ ，He 给出 $\text{Var}[w] = 2/M$ 。多出来的因子 2，正好补偿 ReLU 丢掉的那一半。He 等人的实验表明：对于 30 层的 ReLU 网络，He 初始化可以正常收敛，而 Xavier 初始化的训练完全停滞。

4.5 推广到 PReLU

PReLU: $f(y) = y$ (当 $y > 0$)， $f(y) = ay$ (当 $y \leq 0$)，其中 a 为负半轴斜率。〔PPT 第 49 页〕此时负半轴也有贡献：

$$\mathbb{E}[(a^{(l-1)})^2] = \frac{1}{2}(1+a^2) \text{Var}[z^{(l-1)}] \implies \text{Var}[z^{(l)}] = \frac{1+a^2}{2} M_{l-1} \text{Var}[w^{(l)}] \text{Var}[z^{(l-1)}],$$

令相邻两层方差相等，得

$$\text{Var}[w^{(l)}] = \frac{2}{1+a^2} \cdot \frac{1}{M_{l-1}}. \quad (4.7)$$

检验： $a = 0$ 退化为 He 初始化 (ReLU)； $a = 1$ 为线性激活，得 $1/M_{l-1}$ ，即只考虑前向传播的 Xavier 条件。

4.6 小结

初始化方法	激活函数	均匀分布 $[-r, r]$	高斯分布 $\mathcal{N}(0, \sigma^2)$
Xavier 初始化	Logistic	$r = 4\sqrt{\frac{6}{M_{l-1} + M_l}}$	$\sigma^2 = 16 \times \frac{2}{M_{l-1} + M_l}$
Xavier 初始化	Tanh	$r = \sqrt{\frac{6}{M_{l-1} + M_l}}$	$\sigma^2 = \frac{2}{M_{l-1} + M_l}$
He 初始化	ReLU	$r = \sqrt{\frac{6}{M_{l-1}}}$	$\sigma^2 = \frac{2}{M_{l-1}}$

两个记忆要点：(1) 均匀分布与高斯分布之间用 $r^2/3 = \sigma^2$ 换算，所以 $r = \sqrt{3\sigma^2}$ ；(2) Logistic 函数在 0 附近的斜率为 1/4，信号方差每层会缩小到 1/16，所以方差要乘 16 (r 乘 4) 来补偿。

5 深度神经网络的发展历程

5.1 卷积网络的典型结构

一个**卷积块**为连续 M 个卷积层 (每个后接 ReLU) 和 b 个汇聚层 (M 通常为 $2 \sim 5$ ， b 为 0 或 1)。一个卷积网络由 N 个连续的卷积块堆叠而成，后面再接 K 个全连接层和 softmax (N 的取值范围很大，

如 $1 \sim 100$ 或更大； K 一般为 $0 \sim 2$ ）。〔PPT 第 52 页〕

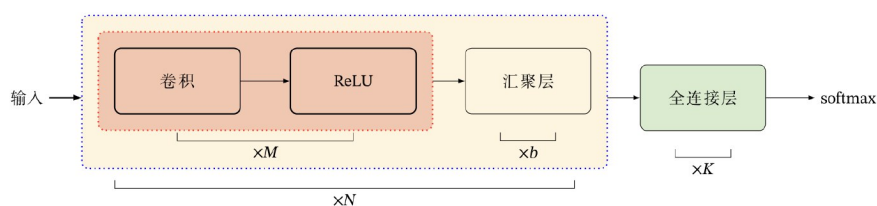


图 12 卷积网络的典型结构。

5.2 ILSVRC：一条“越来越深”的主线

年份	模型	层数	top-5 错误率
2010	Lin et al. (传统方法)	浅层	28.2%
2011	Sánchez & Perronnin (传统方法)	浅层	25.8%
2012	AlexNet (Krizhevsky et al.)	8	16.4%
2013	ZFNet (Zeiler & Fergus)	8	11.7%
2014	VGG (Simonyan & Zisserman)	19	7.3%
2014	GoogLeNet (Szegedy et al.)	22	6.7%
2015	ResNet (He et al.)	152	3.6%
2016	Shao et al.	152	3.0%
2017	SENet (Hu et al.)	152	2.3%
人类水平 (Russakovsky et al.)		—	5.1%

三个节点〔PPT 第 55, 56, 68 页〕：2012 年 AlexNet 是**第一个基于 CNN 的冠军**（ZFNet 可以看作 AlexNet 的调参版）；2014 年网络**更深**（19、22 层）；2015 年起进入“**深度革命**”（152 层），并首次超过人类水平。

5.3 AlexNet (2012)

- ILSVRC 2012 冠军，top-5 错误率约 16%，而第二名为 26%。〔PPT 第 53 页〕
- 第一个现代深度卷积网络模型**，首次使用了许多沿用至今的技术：用 **GPU** 并行训练；采用 **ReLU** 作为激活函数；用 **Dropout** 防止过拟合；使用**数据增强**。
- 结构：5 个卷积层、3 个汇聚层和 3 个全连接层，约 6000 万参数。

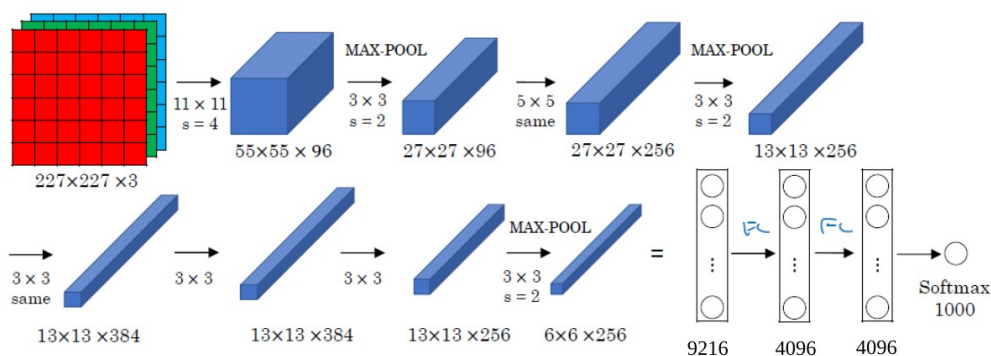


图 13 AlexNet 的结构与各层特征图尺寸。

✎ 输出尺寸的计算

卷积/汇聚层的输出边长为 $\lfloor (W - F + 2P)/S \rfloor + 1$ 。第一层： $(227 - 11)/4 + 1 = 55$ ，输出 $55 \times 55 \times 96$ ；随后 3×3 、步幅 2 的最大汇聚： $(55 - 3)/2 + 1 = 27$ 。依次下去，最后一个汇聚层输出 $6 \times 6 \times 256 = 9216$ 维，再接 $4096 \rightarrow 4096 \rightarrow 1000$ 三个全连接层。〔PPT 第 54 页〕

5.4 VGG (2014)

设计〔PPT 第 57 页〕：更小的核、更小的步幅、更深的网络（8 层 $\rightarrow$ 16–19 层）。全网络**只使用** 3×3 卷积（步幅 1、补 1 圈 0，空间尺寸不变）和 2×2 最大汇聚（步幅 2，尺寸减半）。top-5 错误率从 ZFNet 的 11.7% 降到 7.3%。

为什么用更小的 3×3 卷积核？〔PPT 第 58–59 页〕

1. **感受野相同**：堆叠 3 个 3×3 卷积层的有效感受野与 1 个 7×7 卷积层相同（每多一层，感受野边长加 2： $3 \rightarrow 5 \rightarrow 7$ ）；
2. **非线性更强**：3 层带来 3 次非线性激活，而不是 1 次；
3. **参数更少**：设每层输入、输出均为 C 个通道，则 $3 \times (3^2 C^2) = 27C^2 < 7^2 C^2 = 49C^2$ 。

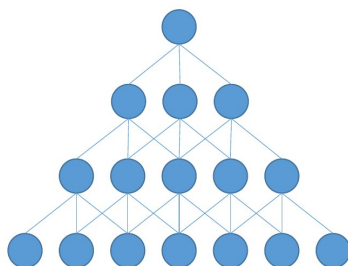


图 14 一维示意：三层 3×3 卷积之后，顶层一个神经元能“看到”底层的 7 个位置。

VGG16 的显存与参数开销〔PPT 第 60 页〕（按卷积块汇总；激活按“特征图元素个数”计，参数忽略偏置）：

模块	输出尺寸	激活个数	参数个数
输入	$224 \times 224 \times 3$	0.15M	0
$2 \times \text{CONV3-64} + \text{POOL}$	$224 \times 224 \times 64 \rightarrow 112^2$	$6.4\text{M} + 0.8\text{M}$	38,592
$2 \times \text{CONV3-128} + \text{POOL}$	$112 \times 112 \times 128 \rightarrow 56^2$	$3.2\text{M} + 0.4\text{M}$	221,184
$3 \times \text{CONV3-256} + \text{POOL}$	$56 \times 56 \times 256 \rightarrow 28^2$	$2.4\text{M} + 0.2\text{M}$	1,474,560
$3 \times \text{CONV3-512} + \text{POOL}$	$28 \times 28 \times 512 \rightarrow 14^2$	$1.2\text{M} + 0.1\text{M}$	5,898,240
$3 \times \text{CONV3-512} + \text{POOL}$	$14 \times 14 \times 512 \rightarrow 7^2$	$0.3\text{M} + 0.025\text{M}$	7,077,888
FC-4096, FC-4096, FC-1000	—	$\approx 0.01\text{M}$	123,633,664
合计		$\approx 15.2\text{M}$	$\approx 138\text{M}$

✓ 两个规律

- **显存大头在前面的卷积层**：特征图的空间尺寸大。仅前向传播，每张图就需要 $15.2\text{M} \times 4$ 字节 $\approx 61\text{MB}$ （课件写作 $24\text{M} \times 4$ 字节 $\approx 96\text{MB}$ ，见第 7 节）；训练时还要保存用于反向传播的中间量，大约再翻倍。
- **参数大头在后面的全连接层**：仅第一个 FC 层就有 $7 \cdot 7 \cdot 512 \cdot 4096 \approx 1.03$ 亿个参数，三个 FC 层合计占总参数的约 89%。

这直接启发了后来的设计：GoogLeNet 和 ResNet 都用全局平均汇聚取代了庞大的全连接层。

5.5 GoogLeNet (2014)

- ILSVRC 2014 冠军，top-5 错误率 6.7%。〔PPT 第 61 页〕
- 更深（22 层），**去掉了庞大的全连接层**，计算效率高：只有约 500 万参数，是 AlexNet（6000 万）的 1/12。
- 由多个 **Inception 模块** 和少量汇聚层堆叠而成。

5.5.1 Inception 模块：不知道用多大的卷积核，那就都用

在卷积网络中，如何设置卷积核的大小是一个十分关键的问题。Inception 模块**同时**使用 1×1 、 3×3 、 5×5 等不同大小的卷积核（对应不同的感受野）以及 3×3 汇聚，再把得到的特征映射**在深度维度上拼接**起来作为输出。〔PPT 第 62 页〕

朴素版本的问题：计算复杂度大。〔PPT 第 63 页〕设模块输入为 $28 \times 28 \times 256$ ，三个卷积分支分别输出 128、192、96 个通道，汇聚分支保持 256 个通道，则输出为 $28 \times 28 \times (128 + 192 + 96 + 256) = 28 \times 28 \times 672$ 。卷积的乘法次数 = 输出元素个数 $\times$ 每个输出元素所需的乘法数：

分支	算式	乘法次数
1×1 conv, 128	$28 \cdot 28 \cdot 128 \cdot 1 \cdot 1 \cdot 256$	25.7M
3×3 conv, 192	$28 \cdot 28 \cdot 192 \cdot 3 \cdot 3 \cdot 256$	346.8M
5×5 conv, 96	$28 \cdot 28 \cdot 96 \cdot 5 \cdot 5 \cdot 256$	481.7M
合计		854M

更糟的是，汇聚分支不改变通道数，拼接之后输出深度只增不减（ $256 \rightarrow 672$ ），逐模块累积下去，计算量会越来越大。

5.5.2 解决办法： 1×1 卷积 bottleneck

在 3×3 、 5×5 卷积**之前**以及汇聚**之后**，各插入一个 1×1 卷积（64 个），把深度从 256 **投影到低维空间**，再做昂贵的空间卷积。〔PPT 第 64–66 页〕

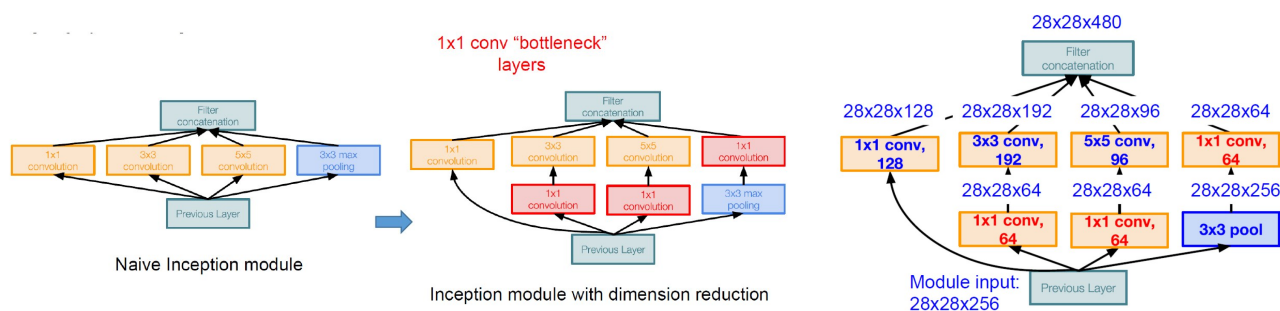


图 15 左：朴素 Inception 模块与带降维的 Inception 模块；右：各分支的特征图尺寸。

卷积	算式	乘法次数
1×1 conv, 64 (3×3 之前)	$28 \cdot 28 \cdot 64 \cdot 1 \cdot 1 \cdot 256$	12.8M
1×1 conv, 64 (5×5 之前)	$28 \cdot 28 \cdot 64 \cdot 1 \cdot 1 \cdot 256$	12.8M
1×1 conv, 128	$28 \cdot 28 \cdot 128 \cdot 1 \cdot 1 \cdot 256$	25.7M
3×3 conv, 192	$28 \cdot 28 \cdot 192 \cdot 3 \cdot 3 \cdot 64$	86.7M
5×5 conv, 96	$28 \cdot 28 \cdot 96 \cdot 5 \cdot 5 \cdot 64$	120.4M
1×1 conv, 64 (汇聚之后)	$28 \cdot 28 \cdot 64 \cdot 1 \cdot 1 \cdot 256$	12.8M
合计		$\approx 271\text{M}$

输出深度也从 672 降到 $128 + 192 + 96 + 64 = 480$ 。(课件上的合计写作 358M，见第 7 节的说明；无论取哪个数，结论都是计算量降到原来的三分之一左右。)

5.5.3 Inception v3

思想与 VGG 一脉相承：用多层的小卷积核替换大卷积核，以减少计算量和参数量。[PPT 第 67 页]

- 用两层 3×3 卷积替换 v1 中的 5×5 卷积（感受野相同，参数 $18C^2 < 25C^2$ ）；
- 用连续的 $n \times 1$ 和 $1 \times n$ 卷积替换 $n \times n$ 卷积（参数 $2n$ 对 n^2 ，论文在 17×17 的特征图上取 $n = 7$ ）。

5.6 ResNet (2015) 与小结

ResNet 的内容见第 2 节。回看整条主线，可以提炼出几个反复出现的设计思想：

✓ CNN 架构设计的共同经验

- 小卷积核堆叠优于单个大卷积核（VGG、Inception v3）；
- 1×1 卷积做通道降维，控制计算量（GoogLeNet、ResNet bottleneck）；
- 去掉庞大的全连接层，改用全局平均汇聚（GoogLeNet、ResNet）；
- 残差连接 + 归一化 + 合适的初始化，让“深度”真正可用（ResNet 及之后几乎所有的深度模型，包括 Transformer）。

6 课后习题与参考解答